

DECISION TREE
AND RANDOM
FOREST

Fundamentals of Machine Learning

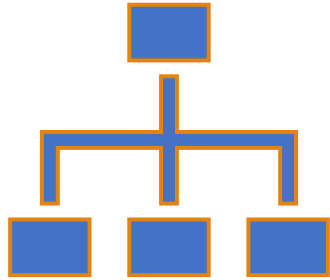
Agenda



Decision Tree



Random Forest



Classification

DECISION TREE

Decision Tree

Key Requirements

Attribute-value Description: Object or case must be expressible in terms of a fixed collection of properties or attributes (e.g: sex, major,...)

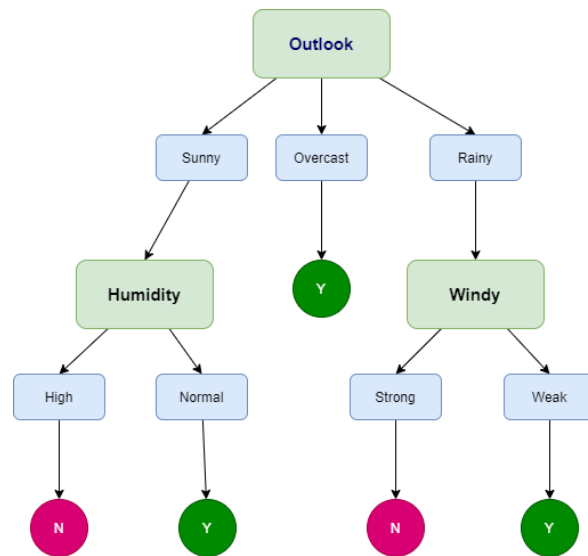
Predefined Classes (Target Values): The target function has discrete output values (Boolean or multiclass)

Sufficient Data: Enough training cases should be provided to learn the model

Decision Tree

Example

Attribute-Value				Target value
Outlook	Temperature	Humidity	Wind	Play
sunny	hot	high	weak	no
sunny	hot	high	strong	no
overcast	hot	high	weak	yes
rainy	mild	high	weak	yes
rainy	cool	normal	weak	yes
rainy	cool	normal	strong	no
overcast	cool	normal	strong	yes
sunny	mild	high	weak	no
sunny	cool	normal	weak	yes
rainy	mild	normal	weak	yes
sunny	mild	normal	strong	yes
overcast	mild	high	strong	yes
overcast	hot	normal	weak	yes
rainy	mild	high	strong	no



Decision Tree

Principled Criterion

Choosing the most useful attribute for classifying examples

Entropy

- A measure of homogeneity of the set of examples
- If the sample is completely homogeneous the entropy is zero and if the sample is an equally divided it has entropy of one

Information Gain

- Measures how well a given attribute separates the training examples according to their target classification
- This measure is used to select among the candidate attributes at each step while growing the tree

Decision Tree

Information Gain

Step 1: Calculate entropy of the target

$$E(S) = - \sum_{c=1}^C \frac{N_c}{N} \log_2 \left(\frac{N_c}{N} \right)$$

Where:

- C : is number of classes
- N_c : is number samples of class c
- N : Total samples

Play	
Yes	No
9	5



$$E(S) = -\frac{9}{14} \log_2 \left(\frac{9}{14} \right) - \frac{5}{14} \log_2 \left(\frac{5}{14} \right)$$

$$E(S) = 0.9403$$

Decision Tree

Information Gain

Step 2: Calculate entropy for each attribute

$$E(x, S) = \sum_{k=1}^K \frac{m_k}{N} E(S_k)$$

Where:

- K : is number values of attribute x

		Play		
		Yes	No	
Outlook	sunny	2	3	5
	overcast	4	0	4
	rainy	3	2	5
Total				14



$$E(\text{Outlook}, S) = \frac{m_{\text{sunny}}}{N} E(S_{\text{sunny}}) + \frac{m_{\text{overcast}}}{N} E(S_{\text{overcast}}) + \frac{m_{\text{rainy}}}{N} E(S_{\text{rainy}})$$

Decision Tree

Information Gain

Step 2: Calculate entropy for each attribute

$$E(\text{Outlook}, S) = \frac{m_{\text{sunny}}}{N} E(S_{\text{sunny}}) + \frac{m_{\text{overcast}}}{N} E(S_{\text{overcast}}) + \frac{m_{\text{rainy}}}{N} E(S_{\text{rainy}})$$

$$E(S_{\text{sunny}}) = -\frac{2}{5} \log_2 \left(\frac{2}{5} \right) - \frac{3}{5} \log_2 \left(\frac{3}{5} \right) = 0.971$$

$$E(S_{\text{overcast}}) = -\frac{4}{4} \log_2 \left(\frac{4}{4} \right) - \frac{0}{4} \log_2 \left(\frac{0}{4} \right) = 0$$

$$E(S_{\text{rainy}}) = -\frac{3}{5} \log_2 \left(\frac{3}{5} \right) - \frac{2}{5} \log_2 \left(\frac{2}{5} \right) = 0.971$$

		Play		
		Yes	No	
Outlook	sunny	2	3	5
	overcast	4	0	4
	rainy	3	2	5
Total				14


$$E(\text{Outlook}, S) = \frac{5}{14} * 0.971 + \frac{4}{14} * 0 + \frac{5}{14} * 0.971 = 0.693$$

Decision Tree

Information Gain

Step 2: Calculate entropy for each attribute

$$E(Outlook, S) = \frac{5}{14} * 0.971 + \frac{4}{14} * 0 + \frac{5}{14} * 0.971 = 0.693$$

$$E(Wind, S) = \frac{8}{14} * 0.5 + \frac{6}{14} * 1.0 = 0.714$$

$$E(Humidity, S) = \frac{7}{14} * 0.985 + \frac{7}{14} * 0.591 = 0.788$$

$$E(Temperature, S) = \frac{4}{14} * 1 + \frac{6}{14} * 0.918 + \frac{4}{14} * 0.811 = 0.910$$

		Play		
		Yes	No	
wind	weak	6	2	8
	strong	3	3	6

		Play		
		Yes	No	
Outlook	sunny	2	3	5
	overcast	4	0	4
	rainy	3	2	5

		Play		
		Yes	No	
humidity	high	3	4	7
	normal	6	1	7

		Play		
		Yes	No	
temperature	hot	2	2	4
	mild	4	2	6
	cool	3	1	4

Decision Tree

Information Gain

Step 3: Calculate Information Gain for each attribute

$$G(x, S) = E(S) - E(x, S)$$

$$G(Outlook, S) = 0.9403 - 0.693 = 0.2473$$

$$G(Wind, S) = 0.9403 - 0.714 = 0.2263$$

$$G(Humidity, S) = 0.9403 - 0.788 = 0.1523$$

$$G(Temperature, S) = 0.9403 - 0.910 = 0.0303$$

Decision Tree

Information Gain

Step 4: Choose attribute with the largest Information Gain as the decision node

$$G(\text{Outlook}, S) = 0.2473$$

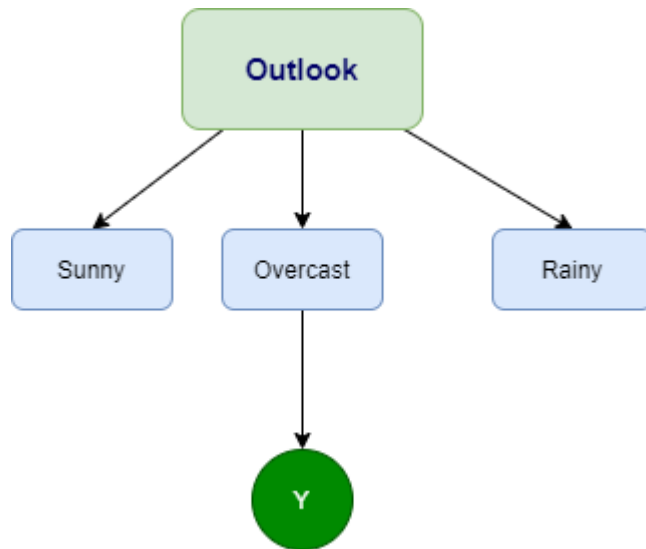
		Play	
		Yes	No
Outlook	sunny	2	3
	overcast	4	0
	rainy	3	2

Decision Tree

Information Gain

Step 5a: A branch with entropy of 0 is a leaf node

		Play	
		Yes	No
Outlook	sunny	2	3
	overcast	4	0
	rainy	3	2

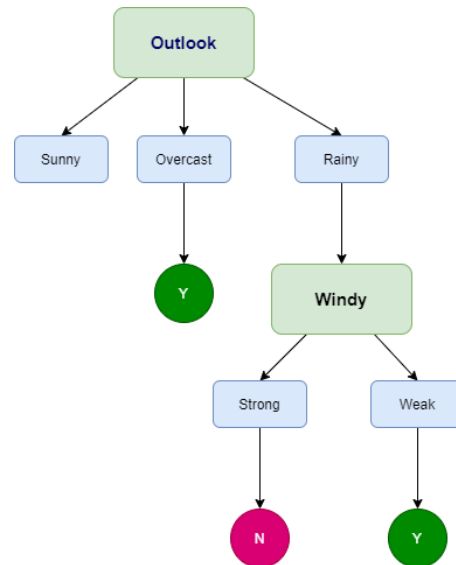


Decision Tree

Information Gain

Step 5b: A branch with entropy more than 0 need further splitting

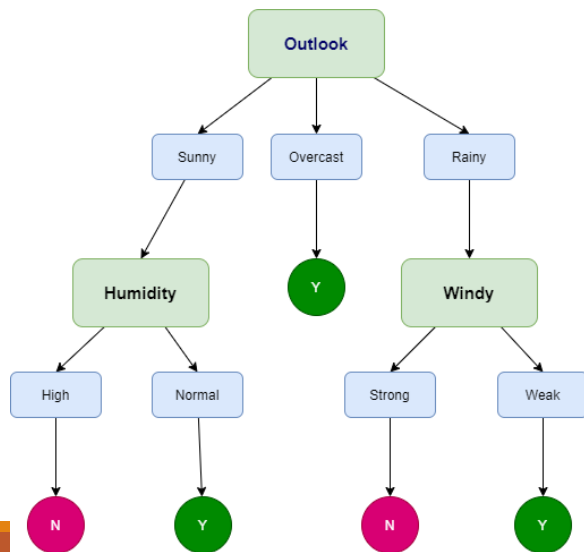
outlook	temperature	humidity	wind	play
rainy	mild	high	weak	yes
rainy	cool	normal	weak	yes
rainy	cool	normal	strong	no
rainy	mild	normal	weak	yes
rainy	mild	high	strong	no



Decision Tree

Information Gain

Step 6: The algorithm is run recursively on the non-leaf branches, until all data is classified



Decision Tree

Decision Tree with sklearn

```
from sklearn import tree
clf = tree.DecisionTreeClassifier(criterion="entropy", max_depth = 4)
```

criterion: The function to measure the quality of a split: gini, entropy

max_depth: The maximum depth of the tree

clf.fit(X,y): Build a decision tree classifier from the training set (X, y).

clf.predict(X): Predict class for X.

Decision Tree

Pre-processing

Sklearn Decision Trees do not handle categorical variables

Need transform to numeric

Attribute-Value				Target value
Outlook	Temperature	Humidity	Wind	Play
sunny	hot	high	weak	no
sunny	hot	high	strong	no
overcast	hot	high	weak	yes
rainy	mild	high	weak	yes
rainy	cool	normal	weak	yes
rainy	cool	normal	strong	no
overcast	cool	normal	strong	yes
sunny	mild	high	weak	no
sunny	cool	normal	weak	yes
rainy	mild	normal	weak	yes
sunny	mild	normal	strong	yes
overcast	mild	high	strong	yes
overcast	hot	normal	weak	yes
rainy	mild	high	strong	no

```
from sklearn import preprocessing
le_windy = preprocessing.LabelEncoder()
le_windy.fit(my_data['wind'].unique())
X[:,3] = le_windy.transform(X[:,3])
# array([1, 0, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0,
0], dtype=object)
```

Decision Tree

PRACTICE

RANDOM FOREST

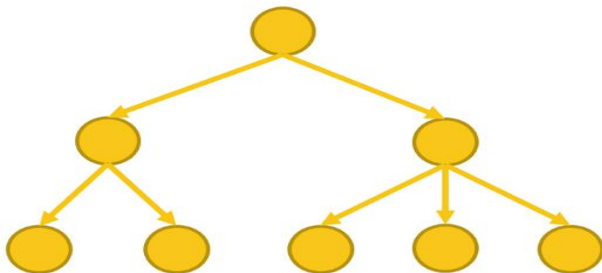
Classification

Random Forest

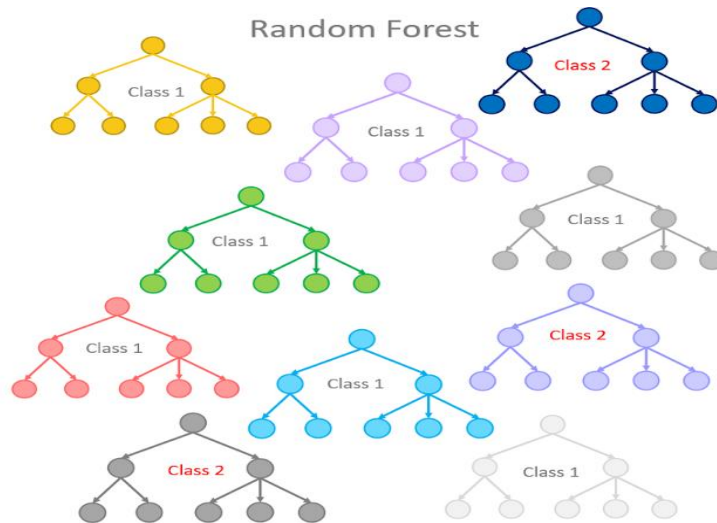
Decision Tree: One tree

Random Forest: More than one tree

Single Decision Tree



Random Forest



Random Forest

Comparsion

Similar to decision tree with a few differences

- For each split-point, the search is not over all variables but just over a part of variables
- No pruning necessary. Trees can be grown until each node contain just very few observations

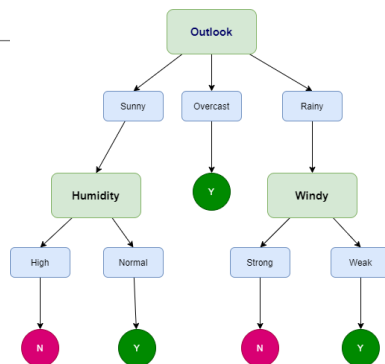
Advantages over decision tree

- Better prediction (in general)
- No parameter tuning necessary with RF

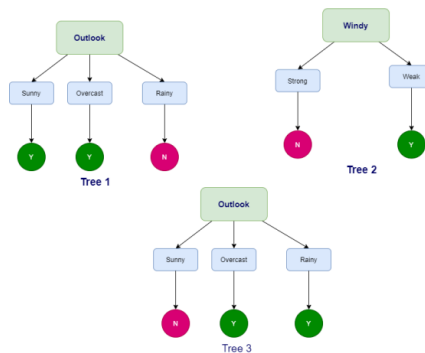
Random Forest

Comparsion

Attribute-Value				Target value
Outlook	Temperature	Humidity	Wind	Play
sunny	hot	high	weak	no
sunny	hot	high	strong	no
overcast	hot	high	weak	yes
rainy	mild	high	weak	yes
rainy	cool	normal	weak	yes
rainy	cool	normal	strong	no
overcast	cool	normal	strong	yes
sunny	mild	high	weak	no
sunny	cool	normal	weak	yes
rainy	mild	normal	weak	yes
sunny	mild	normal	strong	yes
overcast	mild	high	strong	yes
overcast	hot	normal	weak	yes
rainy	mild	high	strong	no



Attribute-Value				Target value
Outlook	Temperature	Humidity	Wind	Play
sunny	hot	high	weak	no
sunny	hot	high	strong	no
overcast	hot	high	weak	yes
rainy	mild	high	weak	yes
rainy	cool	normal	weak	yes
rainy	cool	normal	strong	no
overcast	cool	normal	strong	yes
sunny	mild	high	weak	no
sunny	cool	normal	weak	yes
rainy	mild	normal	weak	yes
sunny	mild	normal	strong	yes
overcast	mild	high	strong	yes
overcast	hot	normal	weak	yes
rainy	mild	high	strong	no

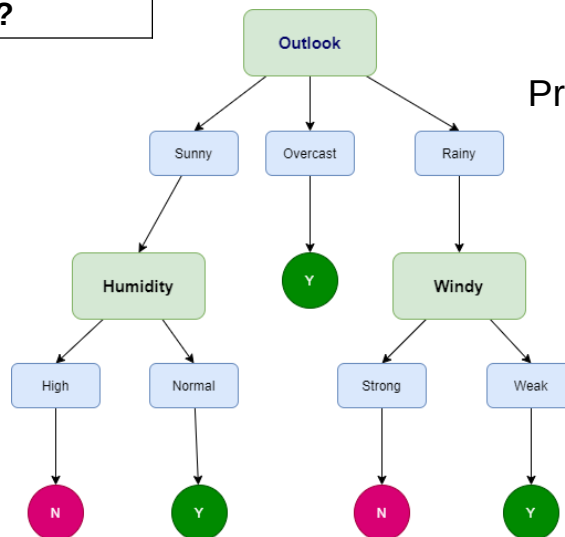


Random Forest

Predict with Decision tree

outlook	temperature	humidity	wind	play
Rainy	mild	high	strong	?

Attribute-Value				Target value
Outlook	Temperature	Humidity	Wind	Play
sunny	hot	high	weak	no
sunny	hot	high	strong	no
overcast	hot	high	weak	yes
rainy	mild	high	weak	yes
rainy	cool	normal	weak	yes
rainy	cool	normal	strong	no
overcast	cool	normal	strong	yes
sunny	mild	high	weak	no
sunny	cool	normal	weak	yes
rainy	mild	normal	weak	yes
sunny	mild	normal	strong	yes
overcast	mild	high	strong	yes
overcast	hot	normal	weak	yes
rainy	mild	high	strong	no



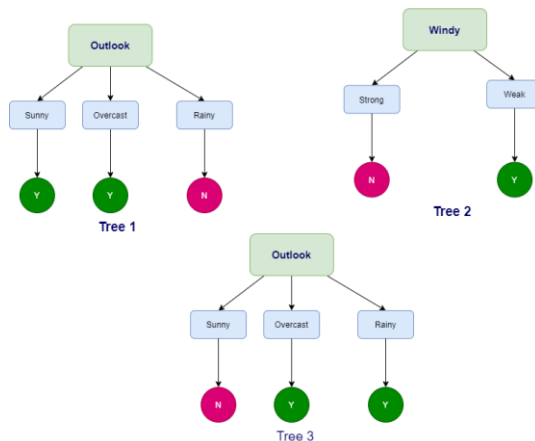
Predict: No

Random Forest

Predict with Decision tree

outlook	temperature	humidity	wind	play
Rainy	mild	high	strong	?

Attribute-Value				Target value
Outlook	Temperature	Humidity	Wind	Play
sunny	hot	high	weak	no
sunny	hot	high	strong	no
overcast	hot	high	weak	yes
rainy	mild	high	weak	yes
rainy	cool	normal	weak	yes
rainy	cool	normal	strong	no
overcast	cool	normal	strong	yes
sunny	mild	high	weak	no
sunny	cool	normal	weak	yes
rainy	mild	normal	weak	yes
sunny	mild	normal	strong	yes
overcast	mild	high	strong	yes
overcast	hot	normal	weak	yes
rainy	mild	high	strong	no



Tree 1: No
Tree 2: No
Tree 3: Yes



No: 2
Yes: 1



Result: No

Decision Tree

Random Forest with sklearn

```
from sklearn.ensemble import RandomForestClassifier  
clf = RandomForestClassifier(n_estimators=100, criterion="entropy", max_depth=5)
```

n_estimators: The number of trees in the forest

criterion: The function to measure the quality of a split: gini, entropy

max_depth: The maximum depth of the tree

clf.fit(X,y): Build a decision tree classifier from the training set (X, y).

clf.predict(X): Predict class for X.

Random Forest

PRACTICE

SUPPORT VECTOR
MACHINES

Fundamentals of Machine Learning

Support Vector Machines

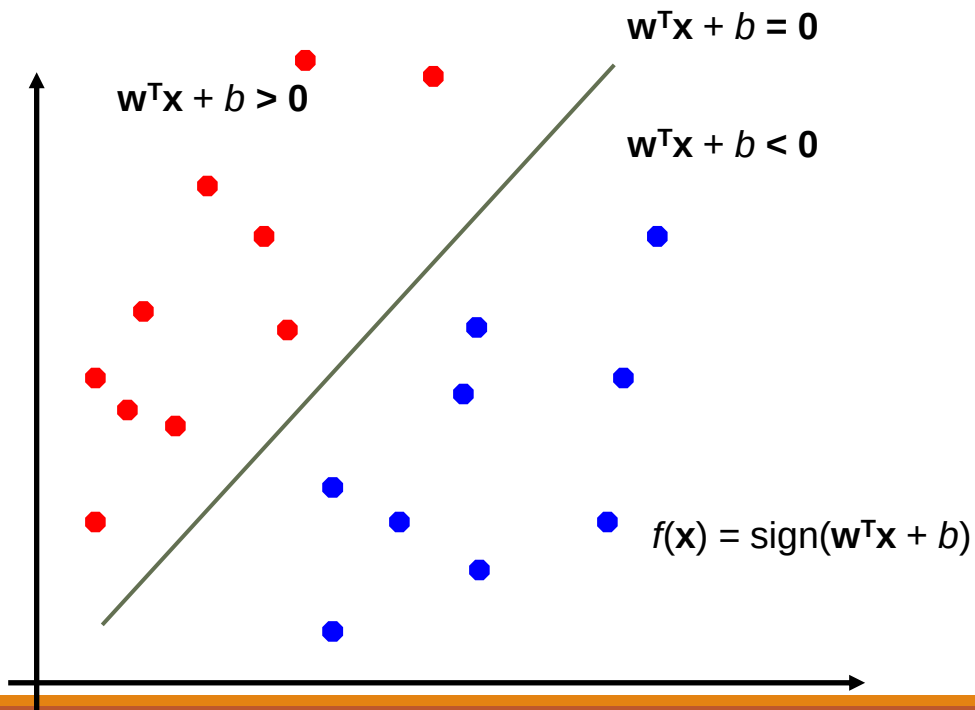
History of SVM

- SVM is related to statistical learning theory
- SVM was first introduced in 1992

Support Vector Machines

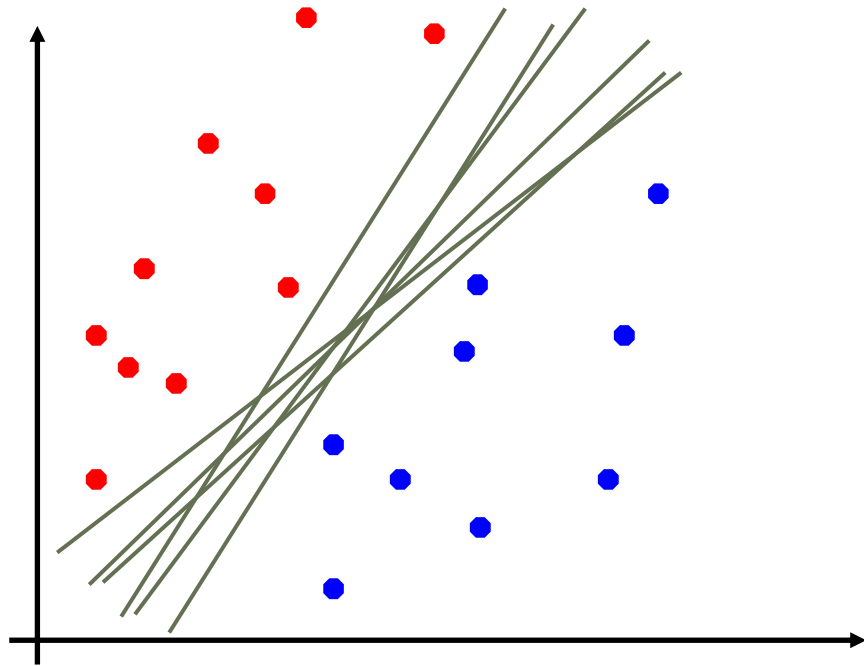
Linear Separators

- Binary classification can be viewed as the task of separating classes in feature space:



Support Vector Machines

Which of the linear separators is optimal?



Support Vector Machines

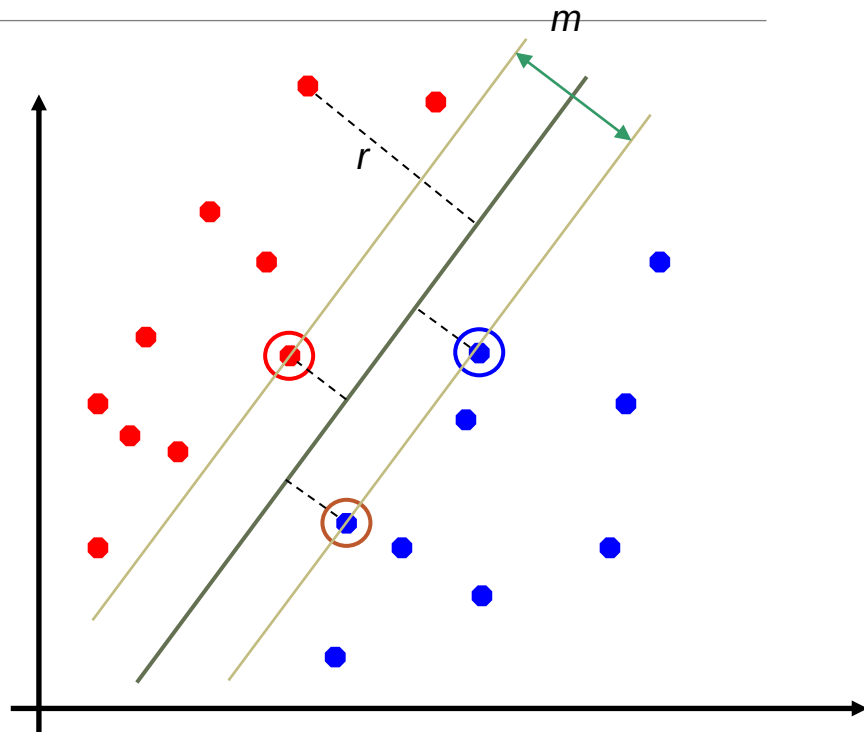
Classification Margin

Distance from example $\mathbf{x}_i$ to the separator is

$$r = \frac{\mathbf{w}^T \mathbf{x}_i + b}{\|\mathbf{w}\|}$$

Examples closest to the hyperplane are **support vectors**.

Margin m of the separator is the distance between support vectors.



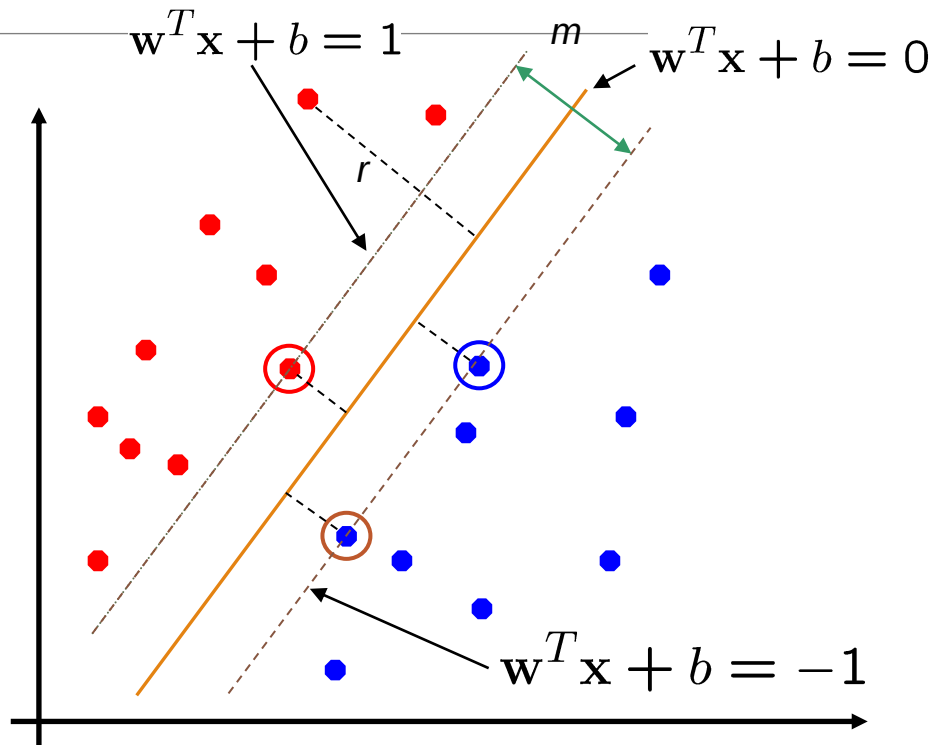
Support Vector Machines

Maximum Margin Classification

The decision boundary should be as far away from the data of both classes as possible

- We should maximize the margin, m
- Distance between the origin and the line $W^T X = -b$ is $\frac{b}{||w||}$

$$m = \frac{2}{||\mathbf{w}||}$$



Support Vector Machines

Finding the Decision Boundary

Let $\{x_1, \dots, x_n\}$ be our data set and let $y_i \in \{1, -1\}$ be the class label of x_i

The decision boundary should classify all points correctly

$$\Rightarrow y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad \forall i$$

To see this: when $y = -1$, we wish $(wx + b) < 1$, when $y = 1$, we wish $(wx + b) > 1$.
For support vectors, we wish $y(wx + b) = 1$.

The decision boundary can be found by solving the following constrained optimization problem

$$\begin{aligned} & \text{Minimize } \frac{1}{2} \|\mathbf{w}\|^2 \\ & \text{subject to } y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \forall i \end{aligned}$$

Support Vector Machines

The Dual Problem

The new objective function is in terms of a_i only

It is known as the dual problem: if we know $\mathbf{w}$, we know all a_i ; if we know all a_i , we know $\mathbf{w}$

The original problem is known as the primal problem

The objective function of the dual problem needs to be maximized!

The dual problem is therefore:

$$\max. W(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1, j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

$$\text{subject to } \alpha_i \geq 0, \quad \sum_{i=1}^n \alpha_i y_i = 0$$

Properties of α_i when we introduce the Lagrange multipliers



The result when we differentiate the original Lagrangian with respect to b

Support Vector Machines

The Dual Problem

$$\begin{aligned} \max. \quad W(\alpha) &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1, j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \\ \text{subject to } \alpha_i &\geq 0, \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

This is a quadratic programming (QP) problem

- A global maximum of α_i can always be found

$\mathbf{w}$ can be recovered by $\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$

Support Vector Machines

Characteristics of the Solution

Many of the α_i are zero (see next page for example)

- $\mathbf{w}$ is a linear combination of a small number of data points
- This “sparse” representation can be viewed as data compression as in the construction of knn classifier

$\mathbf{x}_i$ with non-zero α_i are called support vectors (SV)

- The decision boundary is determined only by the SV
- Let t_j ($j = 1, \dots, s$) be the indices of the s support vectors. We can write

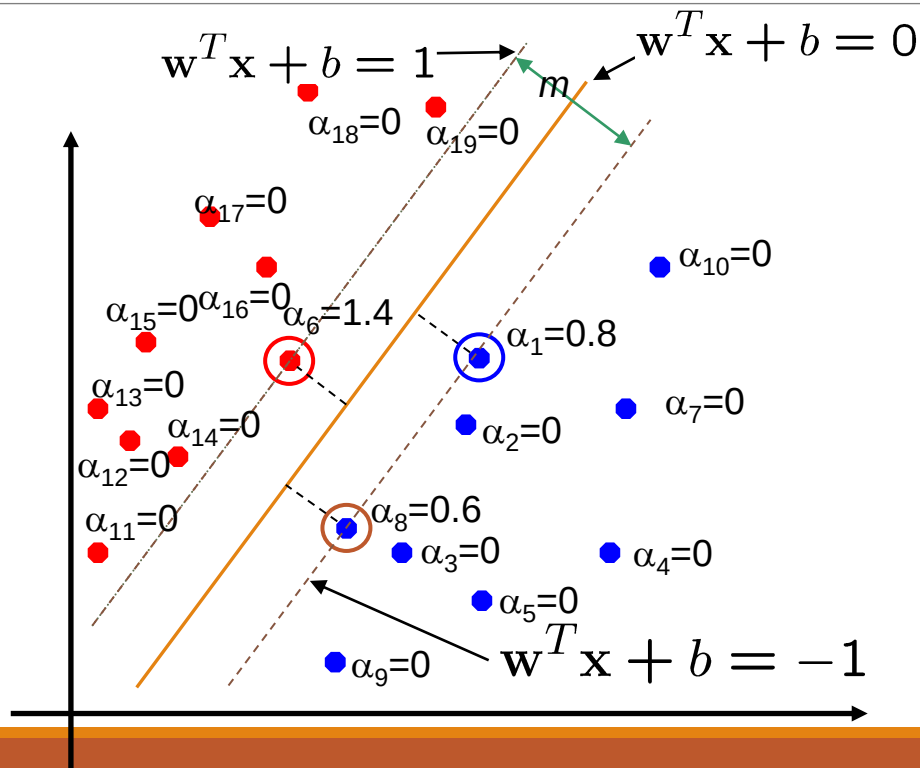
$$\mathbf{w} = \sum_{j=1}^s \alpha_{t_j} y_{t_j} \mathbf{x}_{t_j}$$

For testing with a new data $\mathbf{z}$

- Compute $\mathbf{W}^T \mathbf{z} + \mathbf{b} = \sum_{j=1}^s \alpha_{t_j} y_{t_j} (\mathbf{x}_{t_j}^T \mathbf{z}) + \mathbf{b}$ and classify $\mathbf{z}$ as class 1 if the sum is positive, and class 2 otherwise
- Note: $\mathbf{w}$ need not be formed explicitly

Support Vector Machines

A Geometrical Interpretation

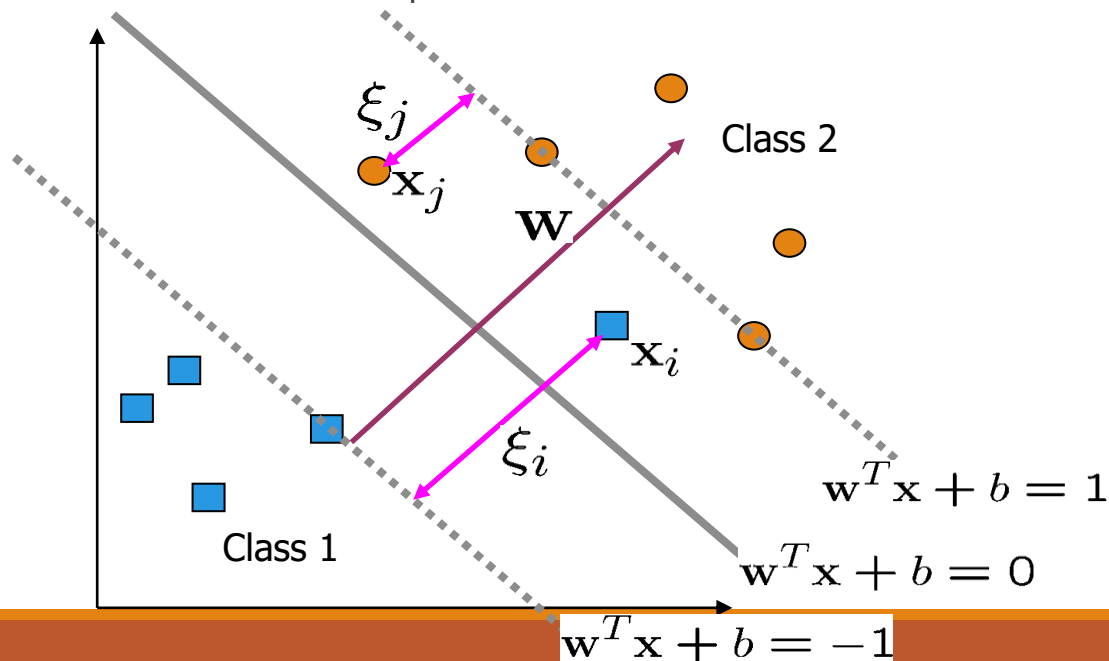


Support Vector Machines

Allowing errors in our solutions

We allow “error” ξ_i in classification; it is based on the output of the discriminant function $\mathbf{w}^T \mathbf{x} + b$

ξ_i approximates the number of misclassified samples



Support Vector Machines

Soft Margin Hyperplane

If we minimize $\sum_i \xi_i$, ξ_i can be computed by

$$\begin{cases} W^T x_i + b \geq 1 - \xi_i \text{ with } y_i = 1 \\ W^T x_i + b \leq -1 + \xi_i \text{ with } y_i = -1 \end{cases}$$

- ξ_i are “slack variables” in optimization
- Note that $\xi_i=0$ if there is no error for $\mathbf{x}_i$
- ξ_i is an upper bound of the number of errors
- We want to minimize $\frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$
 - C: tradeoff parameter between error and margin
- The optimization problem becomes

$$\begin{aligned} & \text{Minimize } \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \\ & \text{subject to } y_i (w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \end{aligned}$$

Support Vector Machines

Extension to Non-linear Decision Boundary

So far, we have only considered large-margin classifier with a linear decision boundary

How to generalize it to become nonlinear?

Key idea: transform $\mathbf{x}_i$ to a higher dimensional space to “make life easier”

- Input space: the space the point $\mathbf{x}_i$ are located
- Feature space: the space of $\boldsymbol{\varphi}(\mathbf{x}_i)$ after transformation

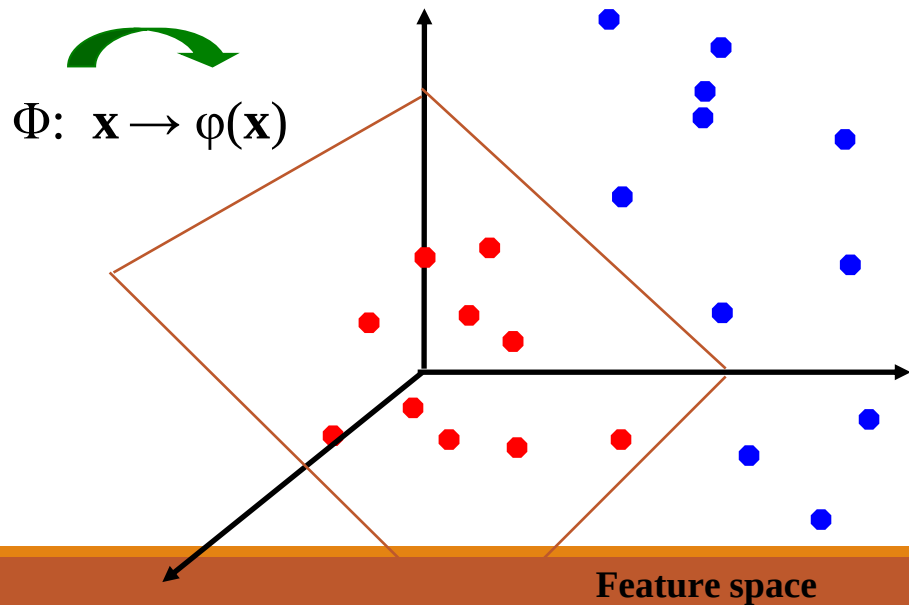
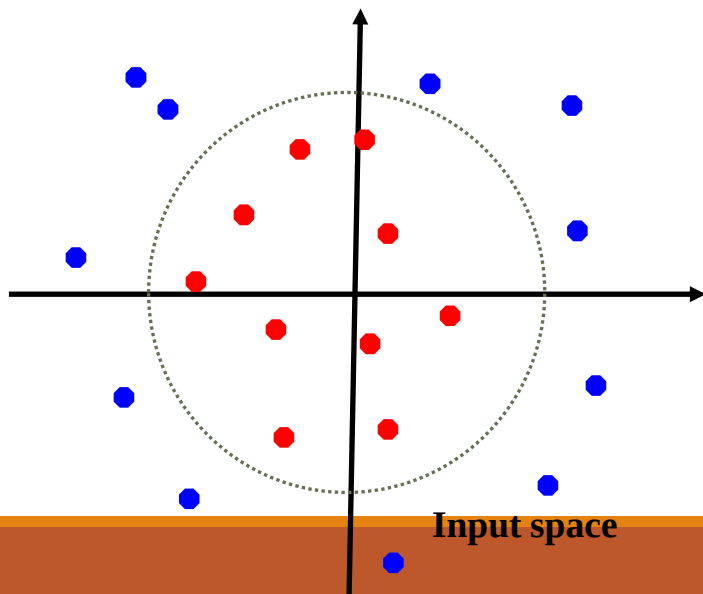
Support Vector Machines

Transforming the Data

Computation in the feature space can be costly because it is high dimensional

- The feature space is typically infinite-dimensional!

The kernel trick comes to rescue



Support Vector Machines

The Kernel Trick

The linear classifier relies on an inner product between vectors $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$

If every data point is mapped into high-dimensional space via some transformation $\Phi: \mathbf{x} \rightarrow \phi(\mathbf{x})$, the inner product becomes:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

A *kernel function* is some function that corresponds to an inner product in some expanded feature space.

Support Vector Machines

An Example for $\varphi(\cdot)$ and $K(\cdot, \cdot)$

Suppose $\varphi(\cdot)$ is given as follows

$$\varphi\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1x_2)$$

An inner product in the feature space is

$$\left\langle \varphi\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right), \varphi\left(\begin{bmatrix} y_1 \\ y_2 \end{bmatrix}\right) \right\rangle = (1 + x_1y_1 + x_2y_2)^2$$

So, if we define the kernel function as follows, there is no need to carry out $\varphi(\cdot)$ explicitly

$$K(x, y) = (1 + x_1y_1 + x_2y_2)^2$$

This use of kernel function to avoid carrying out $\varphi(\cdot)$ explicitly is known as the kernel trick

Support Vector Machines

Examples of Kernel Functions

Polynomial kernel with degree d

$$K(x, y) = (x^T y + 1)^d$$

Radial basis function kernel with width σ

$$K(x, y) = \exp\left(\frac{-\|x - y\|^2}{2\sigma^2}\right)$$

- Closely related to radial basis function neural networks
- The feature space is infinite-dimensional

Sigmoid with parameter k and θ

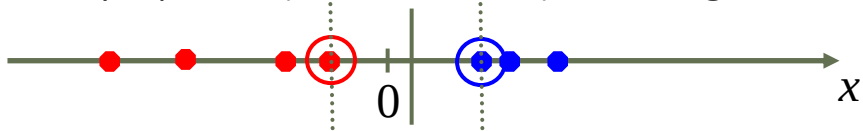
$$K(x, y) = \tanh(kx^T y + \theta)$$

- It does not satisfy the Mercer condition on all k and θ

Support Vector Machines

Examples

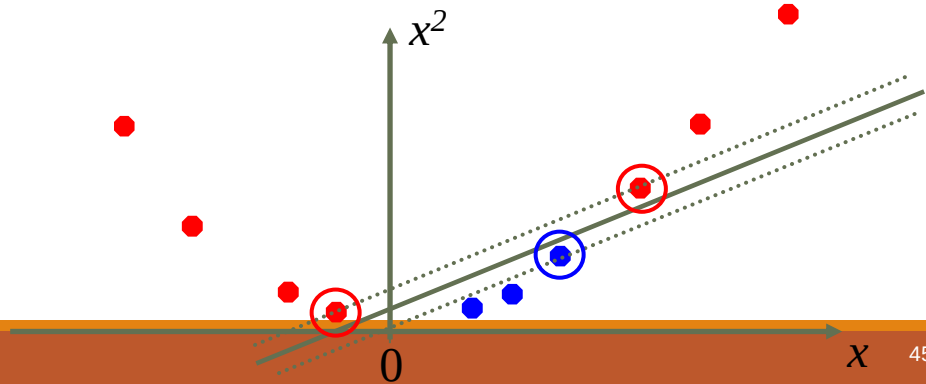
Datasets that are linearly separable (with some noise) work out great:



But what are we going to do if the dataset is just too hard?

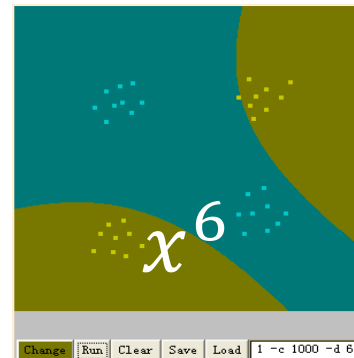
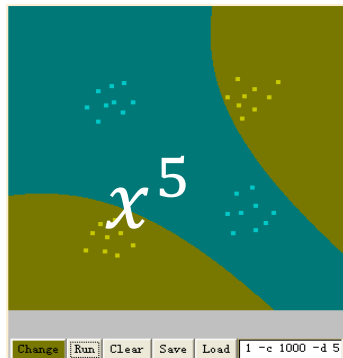
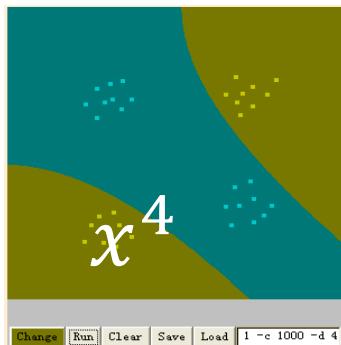
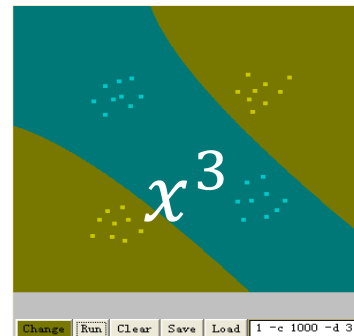
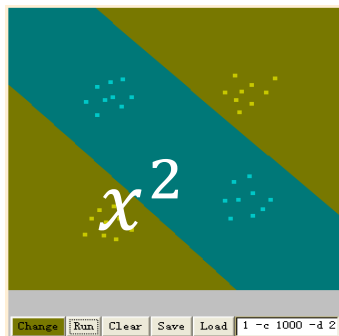
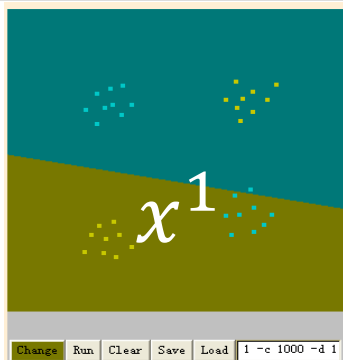


How about ... mapping data to a higher-dimensional space:



Support Vector Machines

Examples



Support Vector Machines

SVM with sklearn

```
from sklearn import svm
clf = svm.SVC(kernel='rbf')
clf.fit(X_train, y_train)
```

➤ Kernel Function in SVC

- Linear
- Polynomial
- Radial basis function (RBF)
- Sigmoid

Support Vector Machines

SVM with sklearn

➤ Kernel Function in SVC

- Linear
- Polynomial
- Radial basis function (RBF)
- Sigmoid

Name	Formula	kernel	Parameters
linear	$x^T y$	'linear'	không có hệ số
polynomial	$(r + \gamma x^T y)^d$	'poly'	d : degree, γ : gamma, r : coef0
sigmoid	$\tanh(\gamma x^T y + r)$	'sigmoid'	γ : gamma, r : coef0
rbf	$\exp(-\gamma \ x - y\ _2^2)$	'rbf'	$\gamma > 0$: gamma



Support Vector Machines

PRACTICE